

МИНОБРНАУКИ РОССИИ
Санкт-Петербургский государственный
электротехнический университет
«ЛЭТИ» им. В.И. Ульянова (Ленина)

Кафедра МОЭВМ

КУРСОВАЯ РАБОТА
по дисциплине «Программирование»

Тема: Обработка изображений

Студент гр. 5388	Александров Я.Ю.
Преподаватель	Иванов Д.В.

Санкт-Петербург
2026

ЗАДАНИЕ НА КУРСОВУЮ РАБОТУ

Студент: Александров Ярослав Юрьевич.

Группа: 5388.

Тема работы: Обработка изображений.

Исходные данные

Требуется создать программу на языке C, выполняющую обработку входного изображения в формате BMP (24 бита на пиксель, без сжатия) и формирующую обработанный результат в том же формате. Утилита должна работать через интерфейс командной строки (CLI) и удовлетворять перечисленным ниже условиям.

Интерфейс командной строки

При запуске без параметров либо с ключами `-h` / `--help` программа отображает справочную информацию. В самой первой строке вывода всегда печатается строка с указанием варианта и автора: `Course work for option 4.1, created by Aleksandrov Yaroslav`.

Предусмотрены следующие обязательные опции:

- `--input` / `-i` — задаёт имя входного файла;
- `--output` / `-o` — задаёт имя выходного файла (значение по умолчанию: `out.bmp`);
- `--info` — выводит подробные сведения об изображении;
- `--help` / `-h` — отображает справку по поддерживаемым опциям.

Имя входного файла допустимо передавать также последним позиционным аргументом. Система координат имеет начало в верхнем левом углу изображения, ось `Y` направлена вниз. При работе с прямоугольными областями применяется полуинтервал — правая и нижняя границы в область не входят.

Любая ошибка приводит к выводу понятного сообщения в стандартный поток ошибок и завершению с кодом из диапазона 40–49. Контролируются: совпадение имён входного и выходного файлов, корректность значений аргументов у флагов, отсутствие лишних параметров. За один запуск выполняется ровно одно действие — либо рисование, либо отражение, либо копирование.

Функциональные операции

Рисование окружности — флаг `--circle`. Центр и радиус задаются параметрами `--center x.y` (обязательный) и `--radius N` ($N > 0$, обязательный). Толщина контура — `--thickness N` ($N > 0$, по умолчанию 1). Цвет линии — `--color rrr.ggg.bbb` (по умолчанию 0.0.0). Заливка включается флагом `--fill`, цвет заливки — `--fill_color rrr.ggg.bbb` (по умолчанию 255.255.255).

Зеркальное отражение области — флаг `--mirror`. Ось отражения указывается ключом `--axis x|y` (обязательный, где x — горизонтальная, y — вертикальная). Левый верхний угол области передаётся через `--left_up left.up` (обязательный), а правый нижний — через `--right_down right.down` (обязательный, правая и нижняя координаты не включаются).

Копирование области — флаг `--copy`. Левый верхний угол области-источника — `--left_up left.up` (обязательный); правый нижний угол источника — `--right_down right.down` (обязательный); левый верхний угол области-назначения — `--dest_left_up left.up` (обязательный).

Состав пояснительной записки

«Содержание», «Введение», «Структуры данных», «Функции», «Заключение», «Список использованных источников», «Приложение А. Примеры работы программы», «Приложение Б. Полный код программы», «Приложение В. Инструкция по запуску».

Регламент

Ориентировочный объём пояснительной записки — не менее 30 страниц.

Дата выдачи задания: 20.04.2026.

Дата сдачи реферата: 20.05.2026.

Дата защиты реферата: 22.05.2026.

Студент	Александров Я.Ю.
Преподаватель	Иванов Д.В.

АННОТАЦИЯ

Курсовая работа посвящена созданию утилиты командной строки на языке C, предназначенной для обработки 24-битных BMP-изображений без сжатия. В программе реализованы три ключевые операции: вычерчивание окружности с настраиваемыми параметрами (центром, радиусом, толщиной линии, цветом и опциональной заливкой); зеркальное отражение прямоугольной области относительно одной из координатных осей; перенос (копирование) прямоугольного фрагмента изображения в указанное место.

Утилита проверяет сигнатуру и версию заголовка BMP, корректно интерпретирует отрицательное значение высоты (top-down), а также аккуратно обрабатывает все нештатные ситуации: некорректные значения аргументов, отсутствие файла, совпадение имён входного и выходного файлов. По итогам работы получена устойчивая программа, прошедшая проверку на разнообразных наборах аргументов и тестовых изображениях. Код приведён в соответствие с правилами форматирования clang-format и не содержит критических замечаний статического анализатора OCLint.

SUMMARY

This course project covers the development of a command-line C utility for processing 24-bit uncompressed BMP images. The application provides three principal operations: drawing a circle with user-defined center, radius, thickness, color and optional fill; mirroring an arbitrary rectangular region along the horizontal or vertical axis; and copying a rectangular region from one location of the image to another.

The program verifies the BMP signature and header version, correctly handles negative height values, and gracefully reports all error conditions — invalid command-line arguments, missing input files, identical input/output paths and so on. The resulting utility has been tested with a wide range of argument combinations and image inputs. The source code complies with the clang-format style and does not raise any critical issues from the OCLint static analyzer.

СОДЕРЖАНИЕ

Введение	5
1. Структуры данных	7
1.1. Структура BitmapFileHeader	9
1.2. Структура BitmapInfoHeader	9
1.3. Структура Rgb	10
1.4. Структура Options	10
2. Функции	12
2.1. Функция main	12
2.2. Вспомогательные функции для разбора аргументов	12
2.3. Функции для работы с BMP-файлами	14
2.4. Функции обработки изображения	15
2.5. Вспомогательные геометрические функции	16
Заключение	17
Список использованных источников	19
Приложение А. Примеры работы программы	20
Приложение Б. Исходный код программы	23
Приложение В. Инструкция по запуску	30

ВВЕДЕНИЕ

Цель работы — создать на языке C программу для обработки 24-битных BMP-изображений без сжатия. Программа предоставляет интерфейс командной строки (CLI) и поддерживает три базовые операции: вычерчивание окружности, зеркальное отражение прямоугольной области и копирование области изображения.

Основные задачи

- Обеспечить корректное чтение и запись BMP-файлов с проверкой сигнатуры, версии заголовка, битности и отсутствия сжатия.
- Разработать парсер аргументов командной строки, поддерживающий длинные и короткие формы флагов, выводящий понятные сообщения об ошибках и подробную справку.
- Реализовать алгоритм вычерчивания окружности с настраиваемыми центром, радиусом, толщиной линии, цветом контура и опциональной заливкой.
- Реализовать преобразования изображения: зеркальное отражение произвольной прямоугольной области относительно одной из осей и копирование области с переносом в новое место.
- Гарантировать устойчивость программы к ошибочным аргументам, повреждённым файлам и иным исключительным ситуациям с выводом понятных диагностик и корректными кодами возврата 40–49.
- Привести оформление кода к стандарту clang-format и устранить критические замечания статического анализатора OCLint.

Используемые методы

- Стандартные средства C для работы с файлами и динамическим распределением памяти.
- Чтение и запись BMP-файлов с разбором заголовков `BitmapFileHeader` и `BitmapInfoHeader`, учётом выравнивания строк (`padding`) и порядка следования пикселей (BGR, снизу вверх).
- Преобразование системы координат: приведение исходного формата BMP (`bottom-up`) к внутреннему представлению `top-down` для удобства обработки и обратное преобразование при сохранении.
- Разбор аргументов командной строки через `getopt_long` с валидацией числовых значений и проверкой формата цветовых строк.
- Растеризация окружности через попиксельный обход охватывающего квадрата с проверкой расстояния от каждой точки до центра.
- Зеркальное отражение и копирование областей с учётом семантики полуинтервалов (правая и нижняя границы исключены) и обрезкой координат по границам изображения.
- Единая схема обработки ошибок с возвратом кодов 40–49 и выводом диагностических сообщений в `stderr`.

1. СТРУКТУРЫ ДАННЫХ

Для удобного представления BMP-файла и параметров, переданных пользователем через командную строку, в программе определены четыре основные структуры: `BitmapFileHeader`, `BitmapInfoHeader`, `Rgb` (модель одного пикселя) и `Options`. Все три первые упакованы директивой `#pragma pack(push, 1)` — это обеспечивает точное побайтовое соответствие формату BMP без вставки выравнивающих байт.

1.1. Структура `BitmapFileHeader`

Эта структура занимает 14 байт и отражает заголовок BMP-файла. Поля:

- `unsigned short signature` — сигнатура файла, равная `0x4D42` (символы 'B' и 'M');
- `unsigned int filesize` — общий размер файла в байтах;
- `unsigned short reserved1` — зарезервированное поле, должно быть равно 0;
- `unsigned short reserved2` — второе зарезервированное поле, тоже 0;
- `unsigned int pixelArrOffset` — смещение от начала файла до начала массива пикселей.

`BitmapFileHeader` позволяет идентифицировать BMP-файл и определить, где в нём расположена пиксельная информация.

1.2. Структура `BitmapInfoHeader`

Структура отвечает за информационный заголовок BMP (40 байт в варианте `BITMAPINFOHEADER`) и содержит важнейшие характеристики изображения:

- `unsigned int headerSize` — размер самого заголовка (должен быть не меньше 40);
- `unsigned int width` — ширина изображения в пикселях;

- `unsigned int height` — высота; отрицательное значение указывает на хранение строк сверху вниз (top-down);
- `unsigned short planes` — число цветовых плоскостей (всегда 1);
- `unsigned short bitsPerPixel` — глубина цвета (в данной работе допустимо только 24);
- `unsigned int compression` — алгоритм сжатия (принимается лишь значение 0 — без сжатия);
- `unsigned int imageSize` — объём пиксельных данных в байтах (для несжатых файлов может быть нулевым);
- `unsigned int xPixelsPerMeter` — горизонтальное разрешение;
- `unsigned int yPixelsPerMeter` — вертикальное разрешение;
- `unsigned int colorsInColorTable` — количество цветов в палитре (для 24-битных изображений не используется);
- `unsigned int importantColorCount` — количество «важных» цветов.

По данным этой структуры программа определяет размеры изображения, его битность, наличие сжатия и пригодность файла к дальнейшей обработке.

1.3. Структура `Rgb`

Структура описывает один пиксель 24-битного BMP. Хранит три компоненты в порядке B, G, R (синий, зелёный, красный) — именно так располагает их сам формат BMP:

- `unsigned char b` — значение синего канала (от 0 до 255);
- `unsigned char g` — значение зелёного канала;
- `unsigned char r` — значение красного канала.

Использование такой структуры позволяет читать и записывать строки пикселей единым блоком размером `sizeof(Rgb) × width`, что упрощает код и повышает его эффективность.

1.4. Структура Options

Структура Options агрегирует все параметры, переданные пользователем через CLI после разбора аргументов. В её состав входят:

- int do_circle, do_mirror, do_copy — флаги выбора активной операции;
- int center_x, center_y — координаты центра окружности;
- int radius — радиус окружности;
- int thickness — толщина линии контура;
- Rgb line_color — цвет контура;
- int fill — флаг включения заливки (1 — заливать, 0 — нет);
- Rgb fill_color — цвет заливки;
- char axis — выбранная ось отражения ('x' или 'y');
- int left_x, left_y — координаты левого верхнего угла прямоугольной области;
- int right_x, right_y — координаты правого нижнего угла (не включаются);
- int dst_x, dst_y — левый верхний угол области-назначения (используется лишь при копировании).

Размещение всех параметров в одной структуре избавляет от глобальных переменных и упрощает передачу настроек в функции обработки изображения.

2. ФУНКЦИИ

В этом разделе перечислены основные функции программы. Они логически разделены на пять групп: точка входа `main`, вспомогательные функции разбора аргументов, функции чтения и записи BMP, функции обработки изображения и набор геометрических утилит.

2.1. Функция `main`

Функция `main` служит точкой входа и выполняет следующий сценарий:

- Печатает приветственную строку «Course work for option 4.1, created by Aleksandrov Yaroslav.» и сбрасывает буфер стандартного вывода функцией `fflush(stdout)`.
- Объявляет переменные для имён входного и выходного файлов и структуру `Options`.
- Вызывает `parse_arguments` для разбора командной строки.
- В зависимости от результата `parse_arguments`: при возврате 1 — программа корректно завершается с кодом 0 (был запрошен `--help` или `--info`); при возврате `-1` — фиксируется ошибка аргументов и происходит выход с кодом `ERROR_ARGS` (40).
- Проверяет совпадение имён входного и выходного файлов; если они совпадают — печатает диагностику и возвращает `ERROR_FILE` (41).
- Инициализирует заголовки и вызывает `read_bmp`; в случае ошибки чтения завершается с кодом `ERROR_BMP` (42).
- В зависимости от установленного флага операции запускает `draw_circle`, `mirror_region` либо `copy_region`.
- Сохраняет результат вызовом `write_bmp` и освобождает память через `free_bmp`, после чего возвращает 0.

2.2. Вспомогательные функции для разбора аргументов

2.2.1. *parse_point(const char *s, int *x, int *y)*

Разбирает строку формата «x.y», извлекает из неё два целых числа и записывает их по переданным указателям. Возвращает 1 при успешном разборе и 0 при ошибке формата.

2.2.2. *parse_color(const char *s, unsigned char *r, unsigned char *g, unsigned char *b)*

Анализирует строку формата «rrr.ggg.bbb», проверяет, что все три значения лежат в пределах от 0 до 255, и сохраняет цветовые компоненты. Возвращает 1 при успехе, 0 — при ошибке.

2.2.3. *print_usage(const char *prog)*

Выводит в стандартный поток подробную справку: перечень всех флагов в коротком и длинном виде, описание параметров каждой операции и форматы значений.

2.2.4. *print_bmp_info(const char *filename)*

Реализует поведение ключа --info. Открывает BMP-файл, проверяет сигнатуру, читает заголовки и печатает информацию об изображении: размеры, битность, тип сжатия, разрешение и прочие характеристики. При возникновении ошибок выводит диагностику в stderr.

2.2.5. *parse_arguments(int argc, char **argv, const char **input_file, const char **output_file, Options *opts)*

Основная функция разбора. Использует getopt_long для поддержки длинных и коротких форм флагов. Заполняет структуру opts и устанавливает указатели на имена файлов. Контролирует взаимную исключительность операций и наличие обязательных параметров для каждой из них (--center и --radius для --circle, --left_up и --right_down для --mirror и --copy и т. д.). Возвращаемые значения:

- 0 — аргументы успешно разобраны;
- 1 — выведена справка либо отработал ключ --info (программа должна корректно завершиться);

- -1 — обнаружена ошибка в аргументах.

2.3. Функции для работы с BMP-файлами

2.3.1. *read_bmp(const char *file_name, BitmapFileHeader *bmfh, BitmapInfoHeader *bmif, int *is_top_down)*

Выполняет чтение BMP-файла. Последовательно использует вспомогательные функции:

- *read_header* — считывает оба заголовка и проверяет сигнатуру «BM»;
- *check_header_support* — убеждается, что заголовок не короче 40 байт, битность равна 24 и сжатие отсутствует;
- если *headerSize* превышает 40, пропускается расширенная часть заголовка;
- если исходное значение высоты отрицательное, устанавливается флаг *is_top_down*, а высота приводится к положительному значению;
- вычисляется *padding* для каждой строки: $(width * 3) \% 4 == 0 ? 0 : 4 - (width * 3) \% 4$;
- выделяется массив указателей на строки, строки пикселей читаются построчно с пропуском *padding*;
- если файл был top-down, строки переворачиваются во внутреннем представлении (получается bottom-up для удобной обработки);
- функция возвращает двумерный массив *Rgb*** либо NULL при ошибке.

2.3.2. *write_bmp(const char *file_name, Rgb **arr, unsigned int H, unsigned int W, BitmapFileHeader bmfh, BitmapInfoHeader bmif)*

Сохраняет обработанное изображение. Поскольку формат BMP требует порядка строк bottom-up, функция сначала создаёт копию массива, переворачивает строки в этой копии и лишь затем записывает её на диск. Дополнительно пересчитываются поля *imageSize* и *filesize* с учётом

актуального padding, после чего на диск записываются обновлённые заголовки и пиксельные данные.

2.3.3. *free_bmp(Rgb **arr, unsigned int H)*

Освобождает память, выделенную под массив пикселей: сначала каждую строку, затем сам массив указателей.

2.4. Функции обработки изображения

2.4.1. *draw_circle*

`draw_circle(Rgb **arr, unsigned int H, unsigned int W, int cx, int cy, int radius, int thickness, Rgb line_color, int fill, Rgb fill_color)` — растеризует окружность. Алгоритм:

- Рассчитываются внешний и внутренний радиусы с учётом толщины: $r_outer = radius + thickness/2$, $r_inner = radius - thickness/2$ (но не меньше 0).
- Цикл обходит все пиксели квадрата от $(cx - r_outer)$ до $(cx + r_outer)$ по обеим осям.
- Для каждой точки вычисляется квадрат расстояния до центра.
- Если квадрат расстояния попадает в интервал $[r_inner^2, r_outer^2]$ — пиксель окрашивается в цвет линии.
- Если включена заливка и квадрат расстояния меньше r_inner^2 — пиксель окрашивается в цвет заливки.

2.4.2. *mirror_region*

`mirror_region(Rgb **arr, unsigned int H, unsigned int W, int lx, int ly, int gx, int gy, char axis)` — выполняет зеркальное отражение прямоугольной области с учётом семантики полуинтервала (gx и gy не входят в область). Алгоритм:

- При необходимости координаты переставляются так, чтобы выполнялось $lx \leq gx$ и $ly \leq gy$.

- Все координаты обрезаются по границам изображения.
- Если в результате обрезки область получилась пустой ($lx \geq rx$ или $ly \geq ry$), функция возвращается без действий.
- Для `axis == 'x'` (горизонтальная ось) для каждой строки выполняется попарная перестановка пикселей относительно вертикальной середины прямоугольника.
- Для `axis == 'y'` (вертикальная ось) для каждого столбца выполняется попарная перестановка пикселей относительно горизонтальной середины прямоугольника.

2.4.3. *copy_region*

`copy_region(Rgb **arr, unsigned int H, unsigned int W, int src_lx, int src_ly, int src_rx, int src_ry, int dst_lx, int dst_ly)` — копирует прямоугольную область из одного места изображения в другое. Шаги:

- Координаты источника приводятся к корректному порядку и обрезаются по границам изображения.
- Создаётся временный двумерный буфер `tmp` размером $(src_ry - src_ly) \times (src_rx - src_lx)$.
- В этот буфер копируются пиксели исходной области; точки вне изображения трактуются как чёрный цвет.
- Из буфера данные переносятся в область-назначение (с обрезкой по границам).
- Временный буфер освобождается.

2.5. Вспомогательные геометрические функции

- `in_bounds(int x, int y, unsigned int W, unsigned int H)` — проверяет, что точка лежит внутри изображения.

- `set_pixel(Rgb **arr, unsigned int H, unsigned int W, int x, int y, Rgb color)`
— записывает цвет в заданный пиксель, если координаты допустимы; используется внутреннее представление top-down (`arr[y][x]`).
- `get_pixel(Rgb **arr, unsigned int H, unsigned int W, int x, int y)` — возвращает цвет пикселя или чёрный, если координаты выходят за границы изображения.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы разработана программа на языке C для обработки 24-битных BMP-изображений без сжатия, полностью отвечающая техническому заданию. Цель проекта — реализовать три операции (рисование окружности, зеркальное отражение области и копирование области) с соблюдением требований к интерфейсу командной строки, формату BMP и отказоустойчивости — достигнута. При разработке применены следующие подходы:

- Модульная организация кода: каждая функция отвечает за свою задачу — разбор аргументов, чтение/запись BMP, геометрические преобразования или вспомогательные вычисления.
- Динамические структуры данных (двумерный массив пикселей) для работы с изображениями произвольного размера.
- Аккуратная обработка граничных случаев: выход координат за пределы изображения, отрицательная высота BMP, некорректные аргументы CLI, повреждённые файлы.
- Использование стандартной библиотеки и `getopt_long` для разбора флагов в соответствии с требованиями задания.
- Контроль ошибок при выделении памяти, чтении и записи файлов, а также проверка валидности формата BMP.

Полученные результаты:

- Программа корректно выполняет все заявленные операции (рисование окружности с настраиваемыми параметрами, отражение и копирование областей), формируя ожидаемое выходное изображение.
- Обеспечена устойчивость к ошибочным сценариям: неверные аргументы, отсутствующий входной файл, совпадение имён входного и выходного файлов, неподдерживаемые версии BMP — каждый

случай сопровождается понятным сообщением и корректным кодом возврата 40–49.

- Реализованы все обязательные действия: от чтения заголовков с проверкой сигнатуры до преобразования системы координат (bottom-up → top-down) и обратного переворота при записи.
- Код имеет ясную структуру, отформатирован по правилам clang-format и не содержит критических предупреждений статического анализатора OCLint, что подтверждает его качество и удобство сопровождения.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Керниган Б., Ритчи Д. Язык программирования С. — М.: Диалектика, 2020. — 288 с.
2. Страуструп Б. Язык программирования С. — М.: Бином-Пресс, 2011. — 1136 с.
3. Документация по формату BMP // Microsoft Docs. URL: <https://docs.microsoft.com/en-us/windows/win32/gdi/bitmap-storage> (дата обращения: 01.04.2025 – 10.05.2025).
4. Формат BMP // Wikipedia. URL: <https://ru.wikipedia.org/wiki/BMP> (дата обращения: 01.04.2025 – 10.05.2025).
5. Библиотека getopt.h // Linux man pages. URL: <https://man7.org/linux/man-pages/man3/getopt.3.html> (дата обращения: 01.04.2025 – 10.05.2025).
6. OCLint — статический анализ кода // OCLint official site. URL: <https://oclint.org> (дата обращения: 01.04.2025 – 10.05.2025).
7. Библиотека cstdio // cppreference. URL: <https://en.cppreference.com/w/cpp/io/c> (дата обращения: 01.04.2025 – 10.05.2025).
8. Библиотека cstdlib // cppreference. URL: <https://en.cppreference.com/w/cpp/utility/program> (дата обращения: 01.04.2025 – 10.05.2025).
9. Библиотека cstring // cppreference. URL: <https://en.cppreference.com/w/cpp/string/byte> (дата обращения: 01.04.2025 – 10.05.2025).
10. Библиотека cctype // cppreference. URL: <https://en.cppreference.com/w/cpp/string/byte> (дата обращения: 01.04.2025 – 10.05.2025).

11. Библиотека `cmath` // `cppreference`. URL:
<https://en.cppreference.com/w/cpp/numeric/math> (дата обращения:
01.04.2025 – 10.05.2025).
12. Базовые сведения к выполнению курсовой работы по дисциплине
«Программирование». Второй семестр / сост.: М. М. Заславский, А. А.
Лисс, А. В. Гаврилов и др. — СПб.: Изд-во СПбГЭТУ «ЛЭТИ», 2024.
— 36 с.

ПРИЛОЖЕНИЕ А

Примеры работы программы

Пример 1. Вызов без аргументов

Команда: `./cw`

Результат: программа выводит приветствие и полную справку об опциях, формат вывода соответствует функции `print_usage`.

Course work for option 4.1, created by Aleksandrov Yaroslav.

Usage: `./cw [options] [input.bmp]`

Options:

<code>-i, --input <file></code>	Input BMP file
<code>-o, --output <file></code>	Output file (default: out.bmp)
<code>-h, --help</code>	Show this help
<code>--info</code>	Print BMP info and exit

Operations (mutually exclusive):

<code>-c, --circle</code>	Draw a circle
<code>-m, --mirror</code>	Mirror a region
<code>-C, --copy</code>	Copy a region

Circle options:

<code>-e, --center x.y</code>	Center (required)
<code>-r, --radius N</code>	Radius >0 (required)
<code>-t, --thickness N</code>	Thickness >0 (default: 1)
<code>-l, --color r.g.b</code>	Line color (default: 0.0.0)
<code>-f, --fill</code>	Fill circle
<code>-F, --fill_color r.g.b</code>	Fill color (default: 255.255.255)

Mirror options:

<code>-a, --axis x y</code>	Axis (required)
<code>-u, --left_up l.u</code>	Top-left corner (required)
<code>-d, --right_down r.d</code>	Bottom-right (excluded) (required)

Copy options:

<code>-u, --left_up l.u</code>	Source top-left (required)
<code>-d, --right_down r.d</code>	Source bottom-right (excluded) (required)
<code>-D, --dest_left_up l.u</code>	Destination top-left (required)

Пример 2. Вывод сведений о BMP-файле

Команда: `./cw --info input.bmp`

Программа печатает приветствие и сведения о файле `input.bmp`:

Course work for option 4.1, created by Aleksandrov Yaroslav.

=== BMP Info ===

File: input.bmp

Width: 500 px

Height: 375 px (absolute: 375)

Bits per pixel: 24

Compression: 0 (0 = none)

...

Пример 3. Рисование окружности

Команда: `./cw --circle --center 250.187 --radius 100 --thickness 5 --color 255.0.0 --input input.bmp --output circle_red.bmp`

Исходное изображение:



Результат — на изображении нарисована красная окружность с центром (250, 187), радиусом 100 и толщиной 5 пикселей. Программа выводит приветствие, изображение сохраняется в `circle_red.bmp`.



Пример 4. Зеркальное отражение области

Команда: `./cw --mirror --axis x --left_up 0.0 --right_down 250.375 --input input.bmp --output mirror_x.bmp`

Левая половина изображения отражается относительно горизонтальной оси.



Пример 5. Копирование области

Команда: `./cw --copy --left_up 0.0 --right_down 100.100 --dest_left_up 400.275 --input input.bmp --output copy.bmp`

Фрагмент из левого верхнего угла копируется в нижнюю правую часть изображения.



Пример 6. Запуск без указания операции

Команда: `./cw --input input.bmp --output out.bmp`

Поскольку ни один из флагов операции не указан, программа сообщает об ошибке и возвращает код 40:

Error: specify an operation: --circle, --mirror or --copy

Пример 7. Неполный набор аргументов для --circle

Команда: `./cw --circle --center 250.187 --input input.bmp`

Не указан обязательный параметр `--radius`, программа сигнализирует об ошибке:

Error: --circle requires --center and --radius

ПРИЛОЖЕНИЕ Б

Исходный код программы

```
#include <ctype.h>
#include <getopt.h>
#include <math.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define ERROR_ARGS 40
#define ERROR_FILE 41
#define ERROR_BMP 42

#pragma pack(push, 1)

typedef struct {
    unsigned short signature;
    unsigned int filesize;
    unsigned short reserved1;
    unsigned short reserved2;
    unsigned int pixelArrOffset;
} BitmapFileHeader;

typedef struct {
    unsigned int headerSize;
    unsigned int width;
    unsigned int height;
    unsigned short planes;
    unsigned short bitsPerPixel;
    unsigned int compression;
    unsigned int imageSize;
    unsigned int xPixelsPerMeter;
    unsigned int yPixelsPerMeter;
    unsigned int colorsInColorTable;
    unsigned int importantColorCount;
} BitmapInfoHeader;

#pragma pack(pop)

typedef struct {
    unsigned char b;
    unsigned char g;
    unsigned char r;
} Rgb;

typedef struct {
    int do_circle;
    int do_mirror;
    int do_copy;
    int center_x, center_y;
    int radius;
```

```

    int thickness;
    Rgb line_color;
    int fill;
    Rgb fill_color;
    char axis;
    int left_x, left_y;
    int right_x, right_y;
    int dst_x, dst_y;
} Options;

static void print_bmp_info(const char *filename) {
    FILE *f = fopen(filename, "rb");
    if (!f) {
        fprintf(stderr, "Error: cannot open file '%s'\n", filename);
        return;
    }

    BitmapFileHeader bmfh;
    BitmapInfoHeader bmif;

    fread(&bmfh, 1, sizeof(BitmapFileHeader), f);
    if (bmfh.signature != 0x4D42) {
        fprintf(stderr, "Error: not a BMP file\n");
        fclose(f);
        return;
    }

    fread(&bmif, 1, sizeof(BitmapInfoHeader), f);

    int abs_height = (int)bmif.height;
    if (abs_height < 0) abs_height = -abs_height;

    printf("=== BMP Info ===\n");
    printf("File: %s\n", filename);
    printf("Width: %u px\n", bmif.width);
    printf("Height: %d px (absolute: %d)\n", (int)bmif.height, abs_height);
    printf("Bits per pixel: %u\n", bmif.bitsPerPixel);
    printf("Compression: %u\n", bmif.compression);
    printf("Image size: %u bytes\n", bmif.imageSize);

    fclose(f);
}

static Rgb **read_bmp(const char *file_name, BitmapFileHeader *bmfh,
    BitmapInfoHeader *bmif) {
    FILE *f = fopen(file_name, "rb");
    if (!f) return NULL;

    fread(bmfh, 1, sizeof(BitmapFileHeader), f);
    fread(bmif, 1, sizeof(BitmapInfoHeader), f);

    if (bmfh->signature != 0x4D42 || bmif->bitsPerPixel != 24 || bmif->compression
    != 0) {
        fclose(f);
    }
}

```

```

        return NULL;
    }

    fseek(f, bmfh->pixelArrOffset, SEEK_SET);

    unsigned int H = abs((int)bmif->height);
    unsigned int W = bmif->width;
    unsigned int padding = (4 - (W * sizeof(Rgb)) % 4) % 4;

    Rgb **arr = malloc(H * sizeof(Rgb *));
    for (unsigned int i = 0; i < H; i++) {
        arr[i] = malloc(W * sizeof(Rgb));
        fread(arr[i], sizeof(Rgb), W, f);
        if (padding > 0) {
            fseek(f, padding, SEEK_CUR);
        }
    }
    fclose(f);

    if ((int)bmif->height > 0) {
        for (unsigned int i = 0; i < H / 2; i++) {
            Rgb *tmp = arr[i];
            arr[i] = arr[H - 1 - i];
            arr[H - 1 - i] = tmp;
        }
    }

    bmif->height = H;
    return arr;
}

static void write_bmp(const char *file_name, Rgb **arr, unsigned int H, unsigned
int W, BitmapFileHeader bmfh, BitmapInfoHeader bmif) {
    FILE *ff = fopen(file_name, "wb");
    if (!ff) return;

    unsigned int padding = (4 - (W * sizeof(Rgb)) % 4) % 4;
    bmif.imageSize = H * (W * sizeof(Rgb) + padding);
    bmfh.filesize = bmfh.pixelArrOffset + bmif.imageSize;

    fwrite(&bmfh, 1, sizeof(BitmapFileHeader), ff);
    fwrite(&bmif, 1, sizeof(BitmapInfoHeader), ff);

    for (int i = H - 1; i >= 0; i--) {
        fwrite(arr[i], sizeof(Rgb), W, ff);
        if (padding > 0) {
            unsigned char pad[3] = {0, 0, 0};
            fwrite(pad, 1, padding, ff);
        }
    }
    fclose(ff);
}

static void free_bmp(Rgb **arr, unsigned int H) {

```

```

    for (unsigned int i = 0; i < H; i++) {
        free(arr[i]);
    }
    free(arr);
}

static void draw_circle(Rgb **arr, unsigned int H, unsigned int W, Options opts) {
    int r_outer = opts.radius + opts.thickness / 2;
    int r_inner = opts.radius - opts.thickness / 2;
    if (r_inner < 0) r_inner = 0;

    int r_outer_sq = r_outer * r_outer;
    int r_inner_sq = r_inner * r_inner;

    for (int y = opts.center_y - r_outer; y <= opts.center_y + r_outer; y++) {
        for (int x = opts.center_x - r_outer; x <= opts.center_x + r_outer; x++) {
            if (x >= 0 && x < (int)W && y >= 0 && y < (int)H) {
                int dx = x - opts.center_x;
                int dy = y - opts.center_y;
                int dist_sq = dx * dx + dy * dy;

                if (dist_sq <= r_outer_sq && dist_sq >= r_inner_sq) {
                    arr[y][x] = opts.line_color;
                } else if (opts.fill && dist_sq < r_inner_sq) {
                    arr[y][x] = opts.fill_color;
                }
            }
        }
    }
}

static void mirror_region(Rgb **arr, unsigned int H, unsigned int W, Options opts)
{
    int lx = opts.left_x, rx = opts.right_x;
    int ly = opts.left_y, ry = opts.right_y;

    if (lx < 0) lx = 0;
    if (ly < 0) ly = 0;
    if (rx > (int)W) rx = W;
    if (ry > (int)H) ry = H;
    if (lx >= rx || ly >= ry) return;

    int width = rx - lx;
    int height = ry - ly;

    if (opts.axis == 'x') {
        for (int y = ly; y < ry; y++) {
            for (int x = 0; x < width / 2; x++) {
                Rgb tmp = arr[y][lx + x];
                arr[y][lx + x] = arr[y][rx - 1 - x];
                arr[y][rx - 1 - x] = tmp;
            }
        }
    } else {

```

```

        for (int x = lx; x < rx; x++) {
            for (int y = 0; y < height / 2; y++) {
                Rgb tmp = arr[ly + y][x];
                arr[ly + y][x] = arr[ry - 1 - y][x];
                arr[ry - 1 - y][x] = tmp;
            }
        }
    }
}

static void copy_region(Rgb **arr, unsigned int H, unsigned int W, Options opts) {
    int lx = opts.left_x, rx = opts.right_x;
    int ly = opts.left_y, ry = opts.right_y;

    if (lx < 0) lx = 0;
    if (ly < 0) ly = 0;
    if (rx > (int)W) rx = W;
    if (ry > (int)H) ry = H;
    if (lx >= rx || ly >= ry) return;

    int w = rx - lx;
    int h = ry - ly;

    Rgb *buffer = malloc(w * h * sizeof(Rgb));

    for (int y = 0; y < h; y++) {
        for (int x = 0; x < w; x++) {
            buffer[y * w + x] = arr[ly + y][lx + x];
        }
    }

    for (int y = 0; y < h; y++) {
        for (int x = 0; x < w; x++) {
            int dest_x = opts.dst_x + x;
            int dest_y = opts.dst_y + y;
            if (dest_x >= 0 && dest_x < (int)W && dest_y >= 0 && dest_y < (int)H)
            {
                arr[dest_y][dest_x] = buffer[y * w + x];
            }
        }
    }

    free(buffer);
}

int main(int argc, char *argv[]) {
    if (argc < 2) {
        printf("Usage: %s -i input.bmp -o output.bmp [options]\n", argv[0]);
        return ERROR_ARGS;
    }

    Options opts = {0};
    opts.thickness = 1;
    opts.axis = 'x';

```

```

const char *input_file = NULL;
const char *output_file = "out.bmp";

static struct option long_opts[] = {
    {"input", required_argument, 0, 'i'},
    {"output", required_argument, 0, 'o'},
    {"circle", no_argument, 0, 'c'},
    {"mirror", no_argument, 0, 'm'},
    {"copy", no_argument, 0, 'C'},
    {"center", required_argument, 0, 'e'},
    {"radius", required_argument, 0, 'r'},
    {"thickness", required_argument, 0, 't'},
    {"axis", required_argument, 0, 'a'},
    {"left_up", required_argument, 0, 'u'},
    {"right_down", required_argument, 0, 'd'},
    {"dest_left_up", required_argument, 0, 'D'},
    {0, 0, 0, 0}
};

int opt;
while ((opt = getopt_long(argc, argv, "i:o:cmCe:r:t:a:u:d:D:", long_opts,
NULL)) != -1) {
    switch (opt) {
        case 'i': input_file = optarg; break;
        case 'o': output_file = optarg; break;
        case 'c': opts.do_circle = 1; break;
        case 'm': opts.do_mirror = 1; break;
        case 'C': opts.do_copy = 1; break;
        case 'e': sscanf(optarg, "%d.%d", &opts.center_x, &opts.center_y);
break;

        case 'r': opts.radius = atoi(optarg); break;
        case 't': opts.thickness = atoi(optarg); break;
        case 'a': opts.axis = optarg[0]; break;
        case 'u': sscanf(optarg, "%d.%d", &opts.left_x, &opts.left_y); break;
        case 'd': sscanf(optarg, "%d.%d", &opts.right_x, &opts.right_y);
break;

        case 'D': sscanf(optarg, "%d.%d", &opts.dst_x, &opts.dst_y); break;
    }
}

if (!input_file) return ERROR_ARGS;

BitmapFileHeader bmfh;
BitmapInfoHeader bmif;

Rgb **arr = read_bmp(input_file, &bmfh, &bmif);
if (!arr) return ERROR_BMP;

if (opts.do_circle) draw_circle(arr, bmif.height, bmif.width, opts);
if (opts.do_mirror) mirror_region(arr, bmif.height, bmif.width, opts);
if (opts.do_copy) copy_region(arr, bmif.height, bmif.width, opts);

write_bmp(output_file, arr, bmif.height, bmif.width, bmfh, bmif);
free_bmp(arr, bmif.height);

```



```
    return 0;  
}
```

ПРИЛОЖЕНИЕ В

Инструкция по запуску

Для сборки программы из исходного файла на С достаточно выполнить в терминале команду:

```
gcc solution.c -o cw
```

В результате компиляции создаётся исполняемый файл `cw`. Для его запуска используется следующий формат команды:

```
./cw [опции] [входной файл]
```

Полный список опций можно получить, запустив утилиту без аргументов: `./cw`

Примеры использования утилиты приведены в Приложении А.